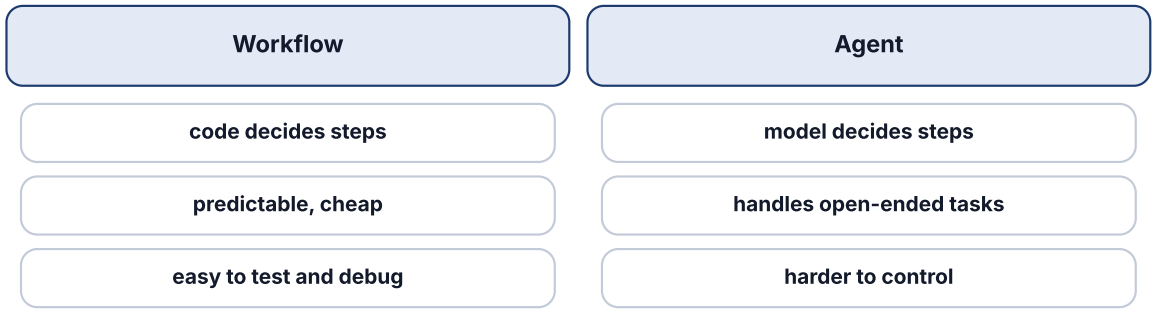
Agents & Orchestration

1. Definitions

- **Agent**: a model in a loop with tools. It gets a goal, picks an action, observes the result, and repeats until done or a stop condition fires. **The model decides the path**
- **Workflow**: a predefined sequence where **code decides the path** and LLM calls fill specific slots (classify, extract, draft)
- **Workflows are the default.** Agents earn their place only when the path genuinely can't be known in advance (open-ended research, operating software whose state must be discovered)

✎ DIAGRAM



'We built an agent' isn't a boast when a five-step workflow would do the job at a tenth of the cost.

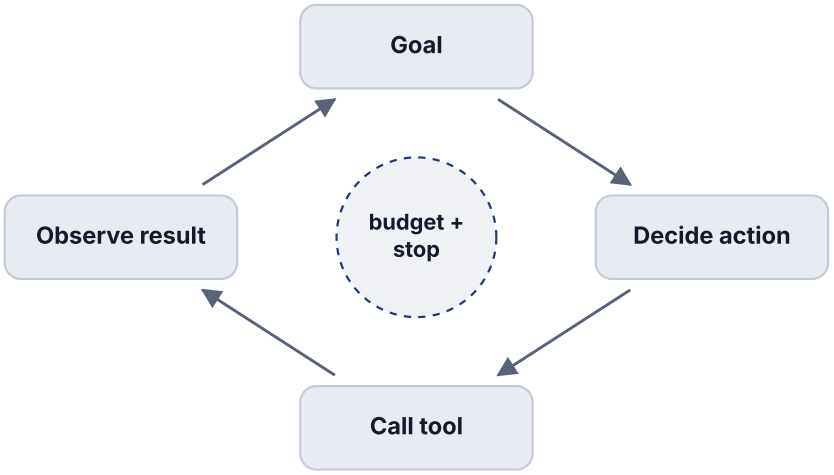
2. Tools

- The model chooses tools by reading their **names and descriptions**, so a vague description is a routing bug. Agent quality is mostly tool quality
- Few tools, sharply described, typed narrow inputs, errors written so the model can act on them
- **Idempotent** tools: safe to call twice, because retries happen
- **MCP** (Model Context Protocol) standardised how tools are plugged in, but every connected server widens the security surface

3. Loop hygiene

- Explicit **budgets**: max steps, tokens, spend
- **Stop conditions** and **checkpoints** that save state
- **Human approval** before anything consequential or irreversible: sending messages, moving money, deleting data, committing code
- Ask of every tool: what's the worst thing this can do, and who approves it?

✎ DIAGRAM

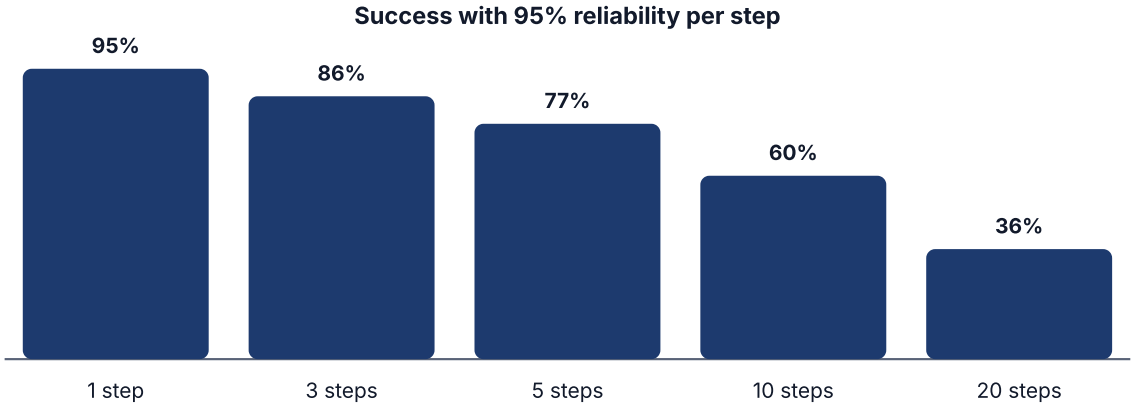


The agent loop. Put a human gate on any arrow that moves money, sends messages or deletes data.

4. Why agents fail on long tasks

- **Compounding error**: 95% reliability per step is only ~60% over ten steps
- Fix with structure, not hope: shorter verified segments, external checkpoints, clean context, capped steps
- Measure **task success rate** on realistic end-to-end tasks, plus cost and steps, and review failed trajectories

✎ DIAGRAM



Errors multiply across steps, which is why long agent runs need checkpoints and verification.

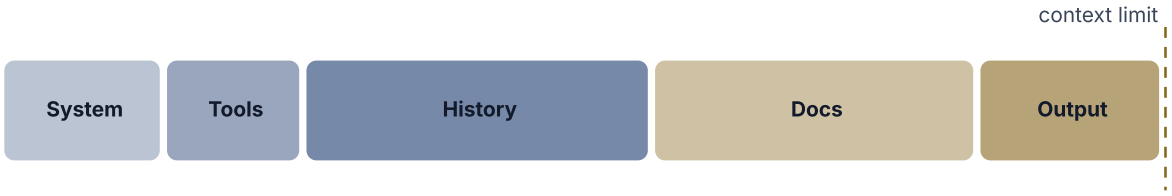
5. Context windows and agent memory

- The **context window** is everything the model sees in one call: system prompt, tools, history, retrieved documents and its own output. Treat it as a **budget**, not a bucket
- Long chats: **sliding window** (last N turns), **summarisation** of older turns, **compaction** (rewrite history shorter); keep stable content at the top for prompt

caching

- Memory types: **short-term** (this task), **long-term** (facts and past episodes retrieved when relevant), **profile** (stable user facts), **procedural** (learned how-to)
- Design questions: who decides what's saved, how wrong memories are corrected, what must never be stored, how one bad memory is stopped from spreading

✎ DIAGRAM



Every block competes for the same window. Summarise history and retrieve only what's relevant.

The context window as a budget.

SAY IT

"I default to a workflow and only use an agent when the steps genuinely can't be known in advance, with step budgets, checkpoints and human approval on anything irreversible."